ELASTICSEARCH WORKSHOP

Category 4: Numeric and Range Queries

EXERCISES

Scenario

You are a data engineer for a fictional intelligence unit called UNIT-7.
Your unit has discovered several weapons caches in the field.
Each cache record has quantities, weights, dates, and risk scores.
You will use range queries to find caches that meet specific criteria.
All locations, quantities, and names are completely fictional.

Key Concept: The range Query

A range query finds documents where a field value is within a given range.
It works on integer, float, double, long, and date fields.

Operator	Meaning	SQL equivalent
`gte`	greater than or equal to	WHERE col >= 30
`gt`	greater than (not including the value)	WHERE col > 30
`lte`	less than or equal to	WHERE col <= 50
`lt`	less than (not including the value)	WHERE col < 50

You can combine operators. Example: gte and lte together means BETWEEN.

```
SQL analogy:     WHERE risk_score BETWEEN 80 AND 95
MongoDB analogy: { risk_score: { $gte: 80, $lte: 95 } }
```

Dataset: Weapons Cache Inventory

Index name: 'cache_inventory' | 12 records to index

cache_id	sector	weapon_type	qty	weight_kg	risk	discovered_on	neutralized
CACHE-01	ALPHA	Rifle	120	340.5	55	2024-01-05	false
CACHE-02	BRAVO	Explosive	45	890.0	92	2024-01-12	false
CACHE-03	ALPHA	Missile	8	1200.0	98	2024-01-18	false
CACHE-04	CHARLIE	Rifle	200	560.0	60	2024-02-02	true
CACHE-05	DELTA	Explosive	30	450.0	85	2024-02-09	false
CACHE-06	BRAVO	Rifle	75	210.5	40	2024-02-14	true
CACHE-07	CHARLIE	Missile	3	780.0	97	2024-02-20	false
CACHE-08	DELTA	Ammunition	5000	95.0	30	2024-03-01	true
CACHE-09	ALPHA	Explosive	60	1100.0	88	2024-03-07	false
CACHE-10	BRAVO	Ammunition	2000	42.0	25	2024-03-11	true
CACHE-11	CHARLIE	Rifle	90	252.0	50	2024-03-15	false
CACHE-12	DELTA	Missile	12	1560.0	99	2024-03-20	false

Exercise 1 [Step-by-step]

Task: Create the 'cache_inventory' index with the correct mapping. Bulk index all 12 records.

Step 1 — Create the index

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

if es.indices.exists(index='cache_inventory'):
    es.indices.delete(index='cache_inventory')

# integer : whole numbers — quantity, risk_score
# float   : numbers with decimals — weight_kg
mapping = {
    'mappings': {
        'properties': {
            'cache_id':      { 'type': 'keyword' },
            'sector':        { 'type': 'keyword' },
            'weapon_type':   { 'type': 'keyword' },
            'quantity':      { 'type': 'integer' },
            'weight_kg':     { 'type': 'float'   },
            'risk_score':    { 'type': 'integer' },
            'discovered_on': { 'type': 'date'    },
            'neutralized':   { 'type': 'boolean' }
        }
    }
}
```

```
es.indices.create(index='cache_inventory', body=mapping)
print('Index created.')
```

Step 2 — Bulk index all 12 records

```
from elasticsearch.helpers import bulk

caches = [
    {'_index':'cache_inventory','_id':'CACHE-01','_source':{
     'cache_id':'CACHE-01','sector':'ALPHA','weapon_type':'Rifle',
     'quantity':120,'weight_kg':340.5,'risk_score':55,
     'discovered_on':'2024-01-05','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-02','_source':{
     'cache_id':'CACHE-02','sector':'BRAVO','weapon_type':'Explosive',
     'quantity':45,'weight_kg':890.0,'risk_score':92,
     'discovered_on':'2024-01-12','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-03','_source':{
     'cache_id':'CACHE-03','sector':'ALPHA','weapon_type':'Missile',
     'quantity':8,'weight_kg':1200.0,'risk_score':98,
     'discovered_on':'2024-01-18','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-04','_source':{
     'cache_id':'CACHE-04','sector':'CHARLIE','weapon_type':'Rifle',
     'quantity':200,'weight_kg':560.0,'risk_score':60,
     'discovered_on':'2024-02-02','neutralized':True}},
    {'_index':'cache_inventory','_id':'CACHE-05','_source':{
     'cache_id':'CACHE-05','sector':'DELTA','weapon_type':'Explosive',
     'quantity':30,'weight_kg':450.0,'risk_score':85,
     'discovered_on':'2024-02-09','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-06','_source':{
     'cache_id':'CACHE-06','sector':'BRAVO','weapon_type':'Rifle',
     'quantity':75,'weight_kg':210.5,'risk_score':40,
     'discovered_on':'2024-02-14','neutralized':True}},
    {'_index':'cache_inventory','_id':'CACHE-07','_source':{
     'cache_id':'CACHE-07','sector':'CHARLIE','weapon_type':'Missile',
     'quantity':3,'weight_kg':780.0,'risk_score':97,
     'discovered_on':'2024-02-20','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-08','_source':{
     'cache_id':'CACHE-08','sector':'DELTA','weapon_type':'Ammunition',
     'quantity':5000,'weight_kg':95.0,'risk_score':30,
     'discovered_on':'2024-03-01','neutralized':True}},
    {'_index':'cache_inventory','_id':'CACHE-09','_source':{
     'cache_id':'CACHE-09','sector':'ALPHA','weapon_type':'Explosive',
     'quantity':60,'weight_kg':1100.0,'risk_score':88,
     'discovered_on':'2024-03-07','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-10','_source':{
     'cache_id':'CACHE-10','sector':'BRAVO','weapon_type':'Ammunition',
     'quantity':2000,'weight_kg':42.0,'risk_score':25,
     'discovered_on':'2024-03-11','neutralized':True}},
    {'_index':'cache_inventory','_id':'CACHE-11','_source':{
     'cache_id':'CACHE-11','sector':'CHARLIE','weapon_type':'Rifle',
     'quantity':90,'weight_kg':252.0,'risk_score':50,
     'discovered_on':'2024-03-15','neutralized':False}},
    {'_index':'cache_inventory','_id':'CACHE-12','_source':{
     'cache_id':'CACHE-12','sector':'DELTA','weapon_type':'Missile',
     'quantity':12,'weight_kg':1560.0,'risk_score':99,
     'discovered_on':'2024-03-20','neutralized':False}},
]
```

```
success, errors = bulk(es, caches)
print('Indexed:', success, '| Errors:', errors)
# Expected: Indexed: 12 | Errors: []
```

Exercise 2 [Step-by-step]

Task: Find all caches with a risk_score of 90 or higher.

Print the cache_id, weapon_type, and risk_score of each result.

Step 1 — Write the range query

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

# range query on a numeric field.
# 'gte': 90 means: find documents where risk_score >= 90.
query = {
    'query': {
        'range': {
            'risk_score': {
                'gte': 90
            }
        }
    }
}
```

Step 2 — Run the search and print results

```
response = es.search(index='cache_inventory', body=query)
hits = response['hits']['hits']

print('Caches with risk >= 90:', len(hits))
for hit in hits:
    src = hit['_source']
    print(src['cache_id'], '-', src['weapon_type'], '- risk:', src['risk_score'])

# Expected output:
# Caches with risk >= 90: 4
# CACHE-02 - Explosive - risk: 92
# CACHE-03 - Missile   - risk: 98
# CACHE-07 - Missile   - risk: 97
# CACHE-12 - Missile   - risk: 99
```

```
Command reference:
  'range': { 'field': { 'gte': value } }   field >= value
  'range': { 'field': { 'gt':  value } }   field >  value
  'range': { 'field': { 'lte': value } }   field <= value
  'range': { 'field': { 'lt':  value } }   field <  value

SQL analogy:     WHERE risk_score >= 90
MongoDB analogy: { risk_score: { $gte: 90 } }
```

Exercise 3

Task: Find all caches where quantity is between 50 and 150 (inclusive on both ends).

Use a range query with both gte and lte together.
Print the cache_id, weapon_type, and quantity of each result.

Expected result: CACHE-01 (120), CACHE-06 (75), CACHE-09 (60), CACHE-11 (90). That is 4 caches.

```
Command reference:
  'range': { 'quantity': { 'gte': 50, 'lte': 150 } }

  Combine gte and lte in one range object. This is the same as SQL BETWEEN.

SQL analogy:     WHERE quantity BETWEEN 50 AND 150
MongoDB analogy: { quantity: { $gte: 50, $lte: 150 } }
```

Exercise 4 (required)

Task: Find all caches where weight_kg is greater than 1000.0 kilograms.

Use gt (not gte). The value 1000.0 itself should NOT be included.
Print the cache_id, weapon_type, and weight_kg of each result.

Expected result: CACHE-03 (1200.0), CACHE-09 (1100.0), CACHE-12 (1560.0). That is 3 caches.

```
Command reference:
  'range': { 'weight_kg': { 'gt': 1000.0 } }
```

```
  gt excludes the boundary value. 1000.0 itself does NOT match.
  gte would include 1000.0 — there is no cache at exactly 1000.0 here, but the rule matters.

SQL analogy:     WHERE weight_kg > 1000.0
```

Exercise 5

Task: Find all caches discovered in February 2024.

February 2024 means: discovered_on >= 2024-02-01 AND discovered_on <= 2024-02-29.
Use a range query on the discovered_on date field.
Print the cache_id, weapon_type, and discovered_on of each result.

Expected result: CACHE-04, CACHE-05, CACHE-06, CACHE-07. That is 4 caches.

```
Command reference:
  'range': {
      'discovered_on': { 'gte': '2024-02-01', 'lte': '2024-02-29' }
  }

  Date values must be strings in YYYY-MM-DD format.
  The same gte / lte / gt / lt operators work on date fields.

SQL analogy:     WHERE discovered_on BETWEEN '2024-02-01' AND '2024-02-29'
```

Exercise 6 (required or 7)

Task: Find all caches discovered in the last 90 days. Use a dynamic date — do not hardcode a date.

Elasticsearch has a built-in keyword called 'now'. It means the current date and time.
'now-90d' means: today minus 90 days.
Use 'now-90d' as the gte value in a range on discovered_on.

Note: the results depend on today's date. Run the query and see what comes back.
Print the cache_id and discovered_on of each result.

```
Command reference:
  'range': { 'discovered_on': { 'gte': 'now-90d' } }

  Date math expressions:
    now       = right now
    now-1d    = 1 day ago
    now-7d    = 7 days ago
    now-1M    = 1 month ago
    now-1y    = 1 year ago
    now/d     = start of today (rounds down to midnight)

  This is useful in dashboards that must always show the last N days.
SQL analogy:     WHERE discovered_on >= CURRENT_DATE - INTERVAL 90 DAY
```

Exercise 7 (required or 6)

Task: Get all 12 caches. Sort them by risk_score from highest to lowest.

Use match_all to get all documents. Add a sort key to order by risk_score descending.
Set size to 12 so all records are returned.
Print the cache_id and risk_score of each result in order.

Expected order: CACHE-12(99), CACHE-03(98), CACHE-07(97), CACHE-02(92), CACHE-09(88), CACHE-05(85), CACHE-04(60), CACHE-01(55), CACHE-11(50), CACHE-06(40), CACHE-08(30), CACHE-10(25).

```
Command reference:
  'query': { 'match_all': {} }          Returns every document. No filter.
  'sort': [ { 'risk_score': { 'order': 'desc' } } ]   Sort highest first.
  'size': 12                            Return all 12. Default is 10.

  Use 'asc' for ascending (lowest first).
  Use 'desc' for descending (highest first).

SQL analogy:     SELECT * FROM cache_inventory ORDER BY risk_score DESC
```

Exercise 8

Task: Find the 3 heaviest caches that have NOT been neutralized. Show only key fields.

Step 1: Filter for neutralized = false using a term query.
Step 2: Sort by weight_kg descending.
Step 3: Return only 3 results.
Step 4: Return only cache_id, weapon_type, weight_kg, and neutralized fields.

Expected result: CACHE-12 (1560.0 kg), CACHE-03 (1200.0 kg), CACHE-09 (1100.0 kg).

```
Command reference — combine all four keys in one query body:
  'query':   { 'term': { 'neutralized': False } }
  'sort':    [ { 'weight_kg': { 'order': 'desc' } } ]
  'size':    3
  '_source': ['cache_id', 'weapon_type', 'weight_kg', 'neutralized']

SQL analogy:
  SELECT cache_id, weapon_type, weight_kg
  FROM cache_inventory WHERE neutralized = false
  ORDER BY weight_kg DESC LIMIT 3
```

Exercise 9

Task: Find all caches where risk_score is between 80 and 95 AND quantity is less than 100.

You cannot stack two range queries at the top level. Use a bool query with must.
Each item in the must list is one condition. All must conditions must be true.
Use this structure:

```
query = {
    'query': {
        'bool': {
            'must': [
                { 'range': { 'risk_score': { 'gte': 80, 'lte': 95 } } },
                { 'range': { 'quantity':   { 'lt': 100 } } }
            ]
        }
    }
}
```

Print the cache_id, risk_score, and quantity of each result.

Expected result: CACHE-02 (risk 92, qty 45), CACHE-05 (risk 85, qty 30), CACHE-09 (risk 88, qty 60).

```
A bool query combines multiple conditions.
```

```
must = ALL conditions must match (like AND in SQL).
You will learn bool queries fully in Category 5.

SQL analogy:     WHERE risk_score BETWEEN 80 AND 95 AND quantity < 100
```

Exercise 10

Task: Paginate all caches sorted by discovered_on ascending. Show 4 per page. Fetch page 2.

Use 'from' and 'size' for pagination.
'size' = results per page.
'from' = how many results to skip before starting.
To get page 2 with 4 per page: skip the first 4. So from = 4, size = 4.

Print the cache_id and discovered_on of the 4 results.

Expected result (sorted by date asc, page 2): CACHE-05, CACHE-06, CACHE-07, CACHE-08.

```
Command reference:
  'from': 4                              Skip the first 4 results.
  'size': 4                              Return the next 4.
  'sort': [ { 'discovered_on': { 'order': 'asc' } } ]

  Page formula:  from = (page_number - 1) x page_size
    Page 1: from = 0
    Page 2: from = 4
    Page 3: from = 8

SQL analogy:     ORDER BY discovered_on ASC LIMIT 4 OFFSET 4
MongoDB analogy: .sort({ discovered_on: 1 }).skip(4).limit(4)
```

End of Category 4 Exercises